JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.function

Interface BiFunction<T,U,R>

Type Parameters:

`T - the type of the first argument to the function`

`U - the type of the second argument to the function`

`R - the type of the result of the function`

All Known Subinterfaces:

`BinaryOperator`<T>

Functional Interface:

`This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.`

```
@FunctionalInterface
public interface BiFunction<T,U,R>
```

Represents a function that accepts two arguments and produces a result. This is the two-arity specialization of `Function`.

This is a functional interface whose functional method is `apply(Object, Object)`.

Since:

1.8

See Also:

`Function`

Method Summary

Abstract Methods | Default Methods

Modifier and Type	Method	Description
default <V> BiFunction<T,U,V>	andThen(Function<? super R,? extends V> after)	Returns a composed function that first applies this function to its input, and then applies the after function to the result.
R	apply(T t, U u)	Applies this function to the given arguments.

Method Detail

apply

```
R apply(T t,
        U u)
```

Applies this function to the given arguments.

Parameters:

t - the first function argument

u - the second function argument

Returns:

the function result

andThen

```
default <V> BiFunction<T,U,V> andThen(Function<? super R,? extends V> after)
```

Returns a composed function that first applies this function to its input, and then applies the after function to the result. If evaluation of either function throws an exception, it is relayed to the caller of the composed function.

Type Parameters:

V - the type of output of the after function, and of the composed function

Parameters:

after - the function to apply after this function is applied

Returns:

a composed function that first applies this function and then applies the after function

Throws:

NullPointerException - if after is null

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.